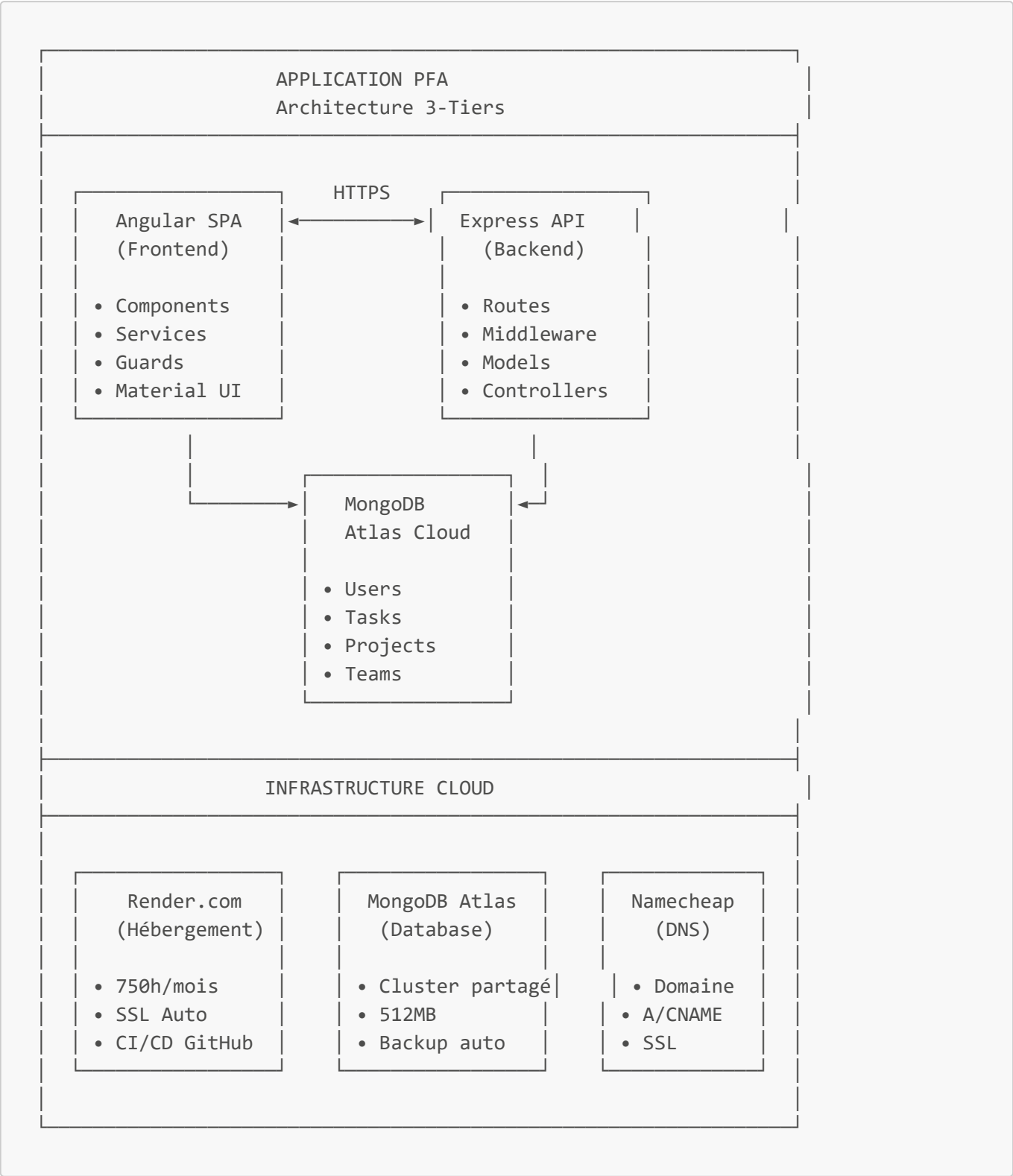


IMAGES EXPLICATIVES - RAPPORT PFA

DIAGRAMME 1: Architecture Générale du Système

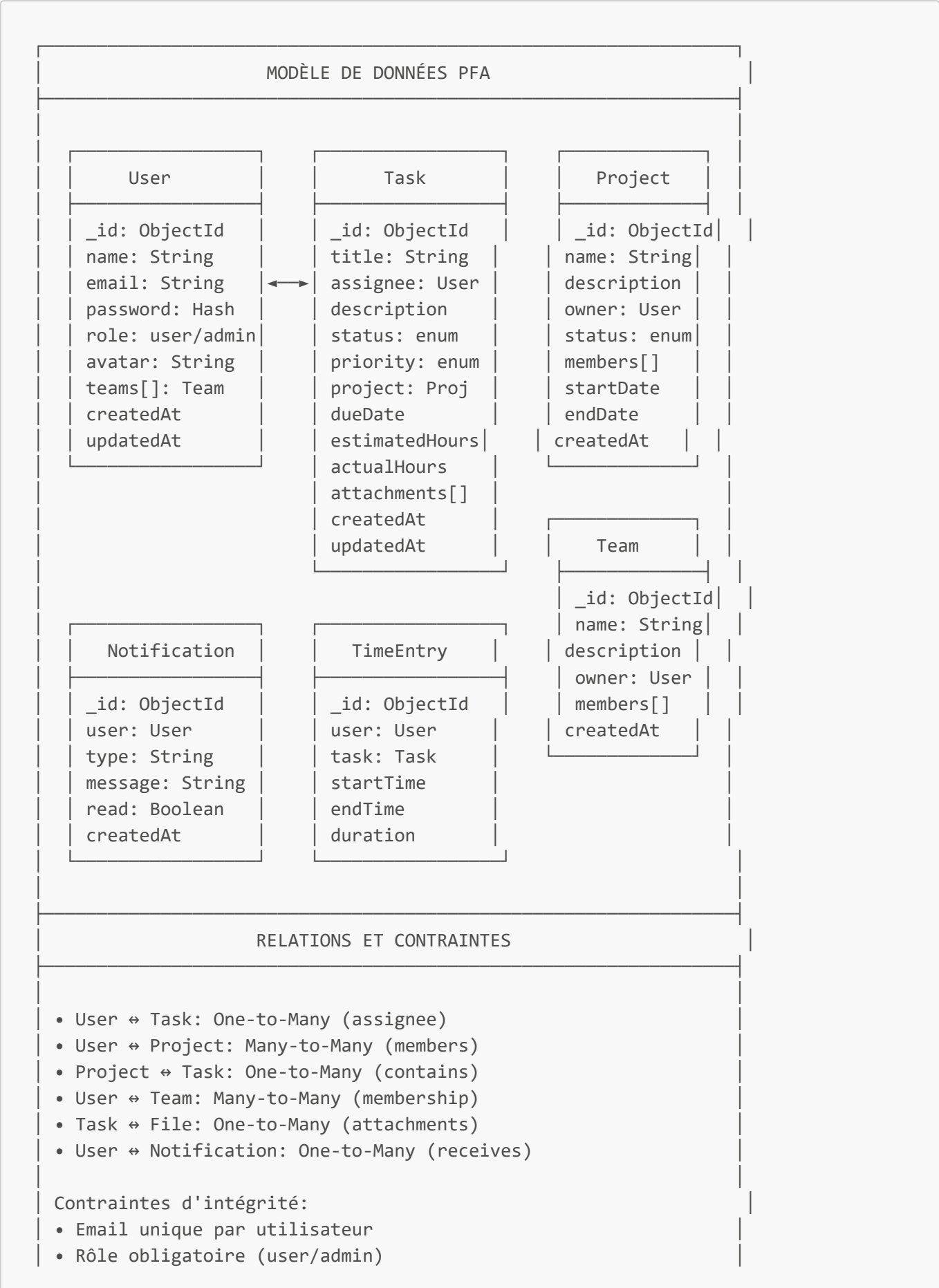


Légende:

- **Frontend:** Interface utilisateur Angular avec Material Design
- **Backend:** API REST Node.js/Express avec authentification JWT
- **Database:** MongoDB Atlas pour persistance des données

- **Infrastructure:** Services cloud gérés et scalables

DIAGRAMME 2: Modèle de Données (Schema MongoDB)

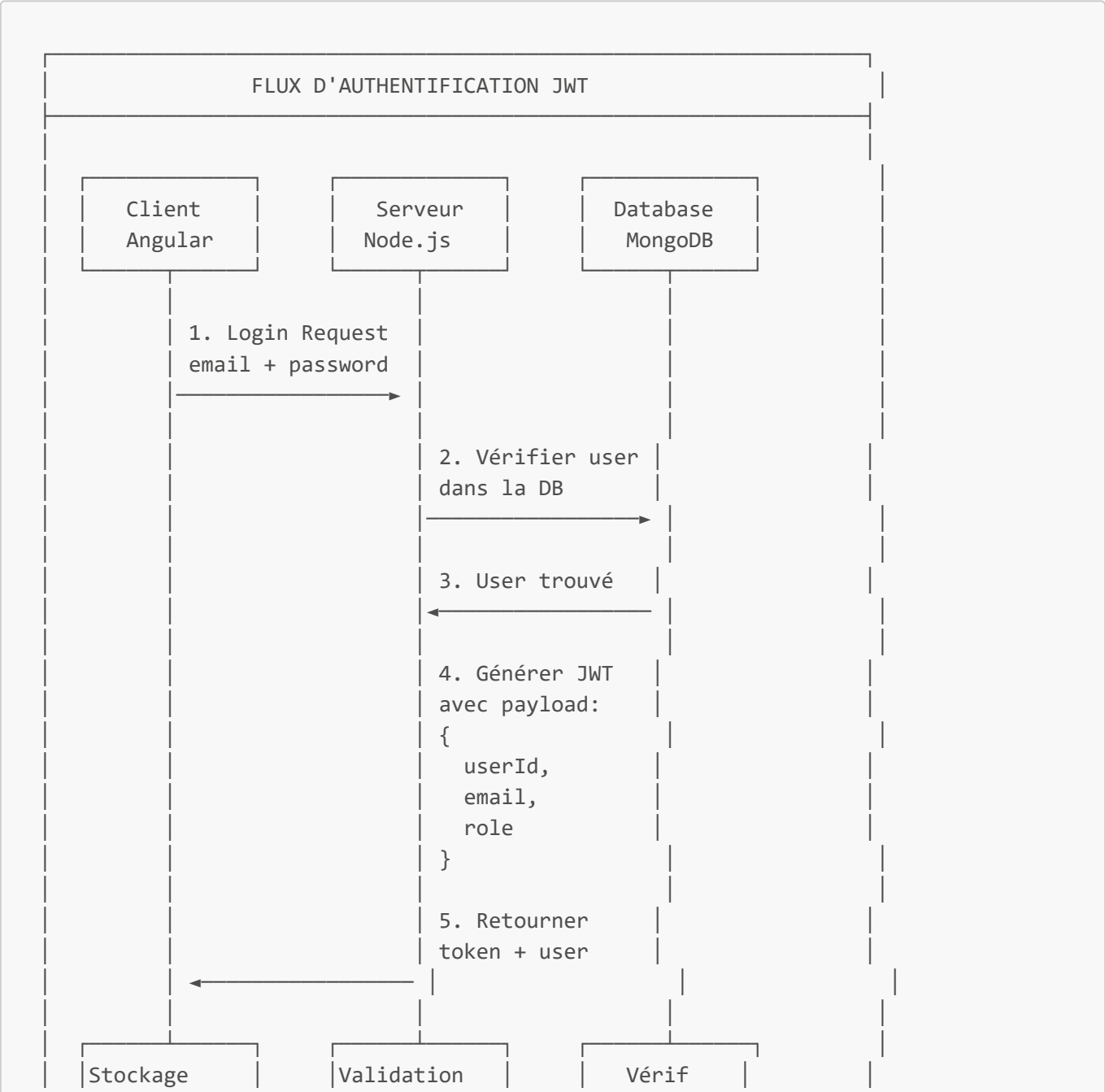


- Status énuméré pour Task/Project
- Dates de cohérence (start < end)

Explication:

- **User:** Entité centrale avec authentification et rôles
- **Task:** Cœur métier avec statut, priorité, et pièces jointes
- **Project:** Conteneur de tâches avec gestion d'équipe
- **Team:** Groupement d'utilisateurs pour collaboration
- **Notification:** Système de messagerie interne
- **TimeEntry:** Suivi du temps passé sur les tâches

DIAGRAMME 3: Flux d'Authentification

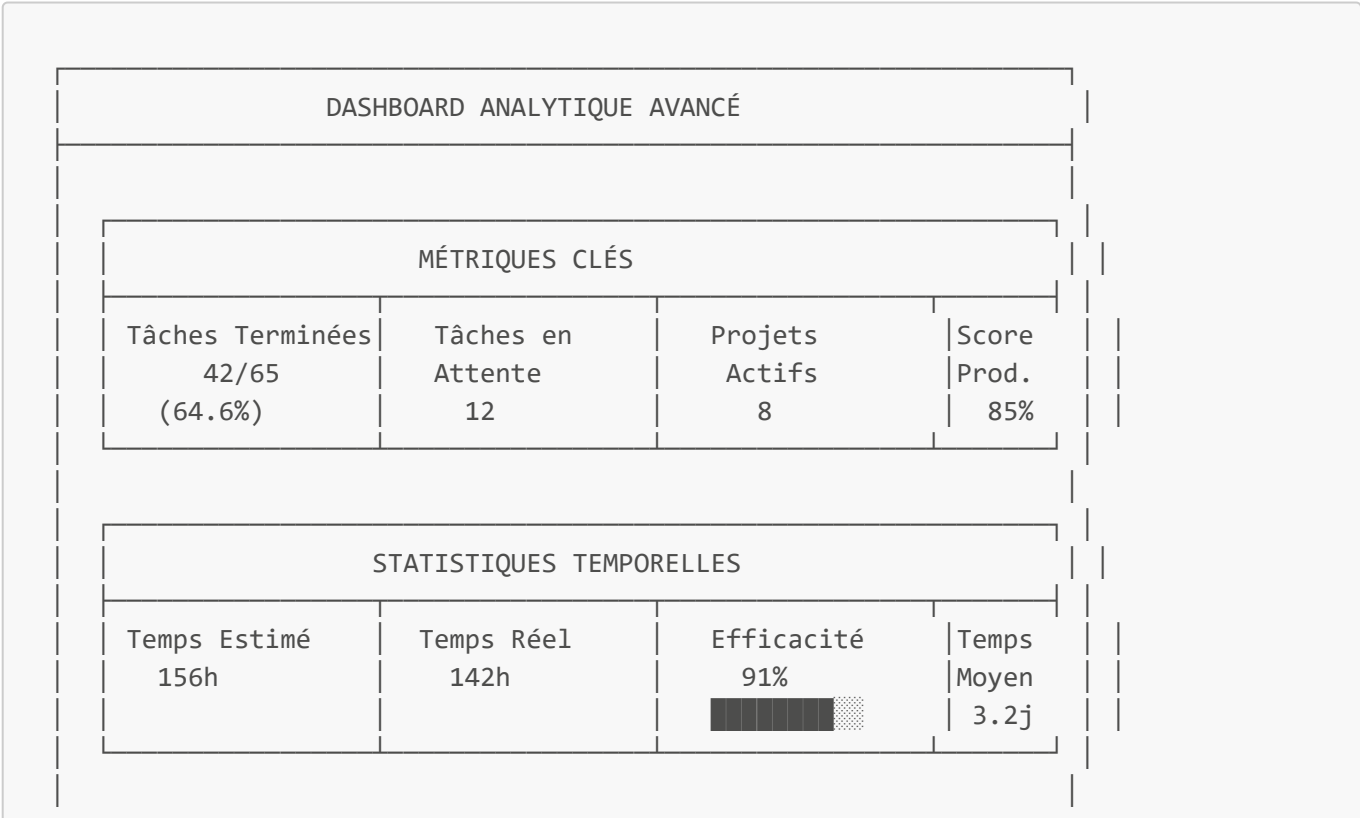


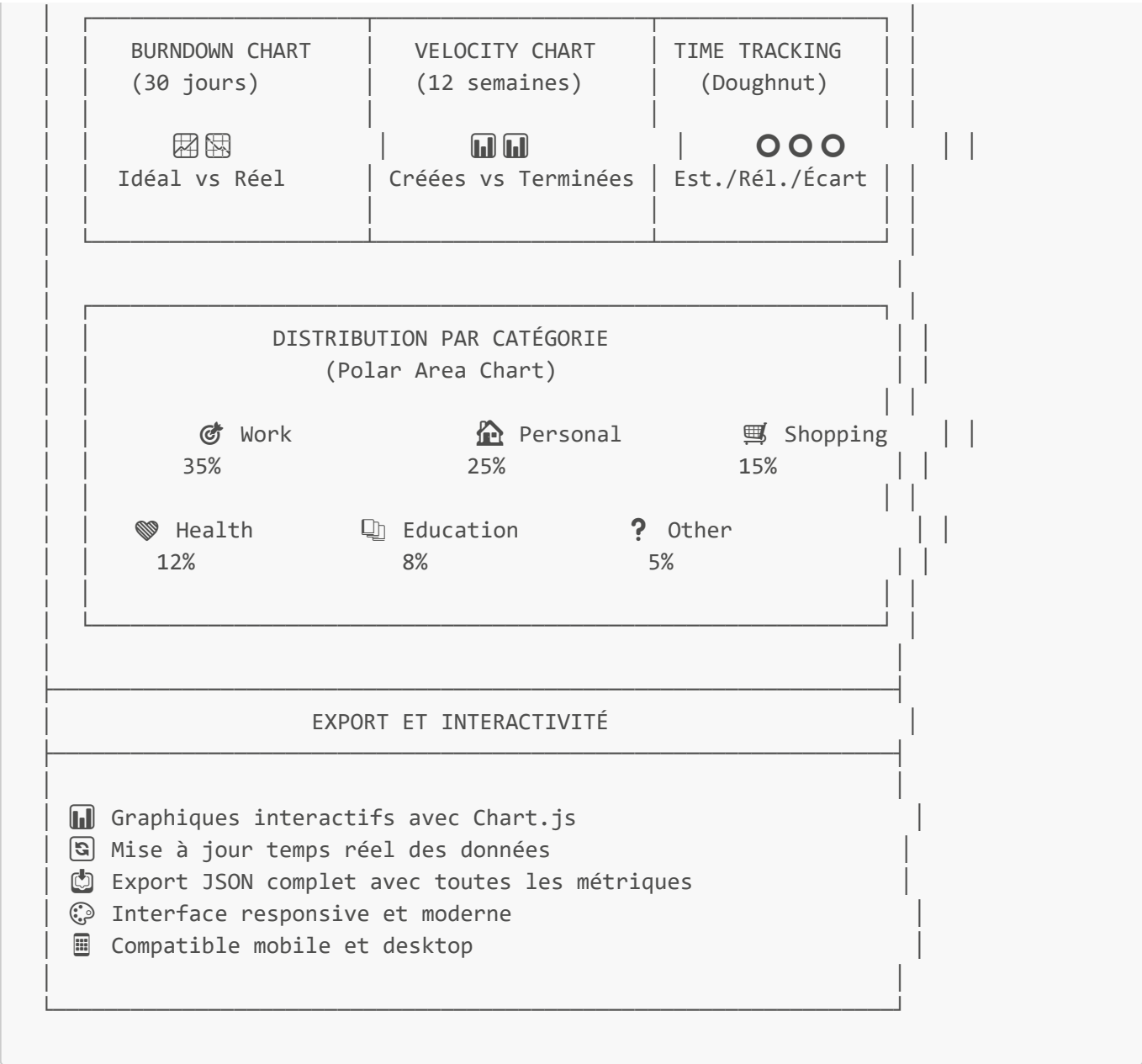


Étapes du processus:

1. **Client** envoie credentials (email, password, role)
2. **Serveur** vérifie dans MongoDB
3. **Succès** → génération JWT avec payload sécurisé
4. **Client** stocke token et informations utilisateur
5. **Requêtes suivantes** incluent le token dans headers

DIAGRAMME 4: Dashboard Analytics

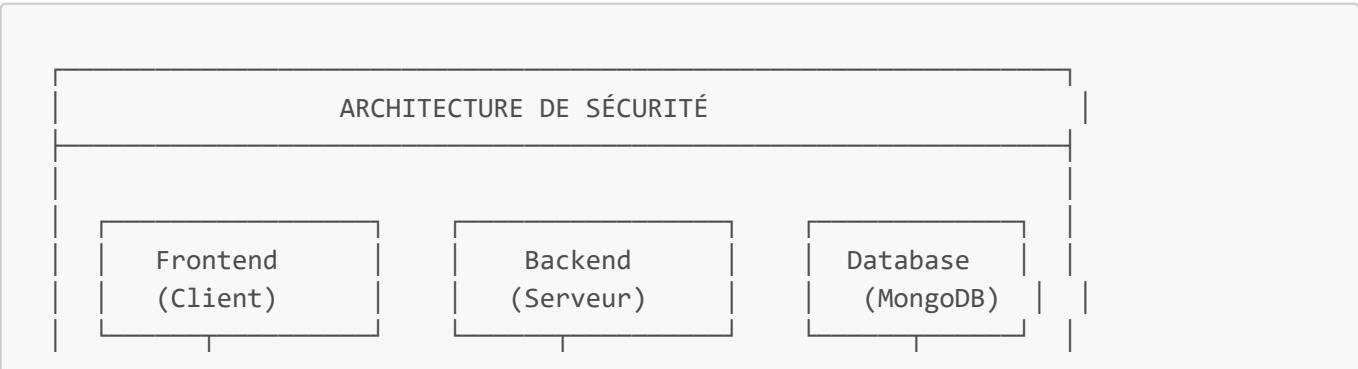


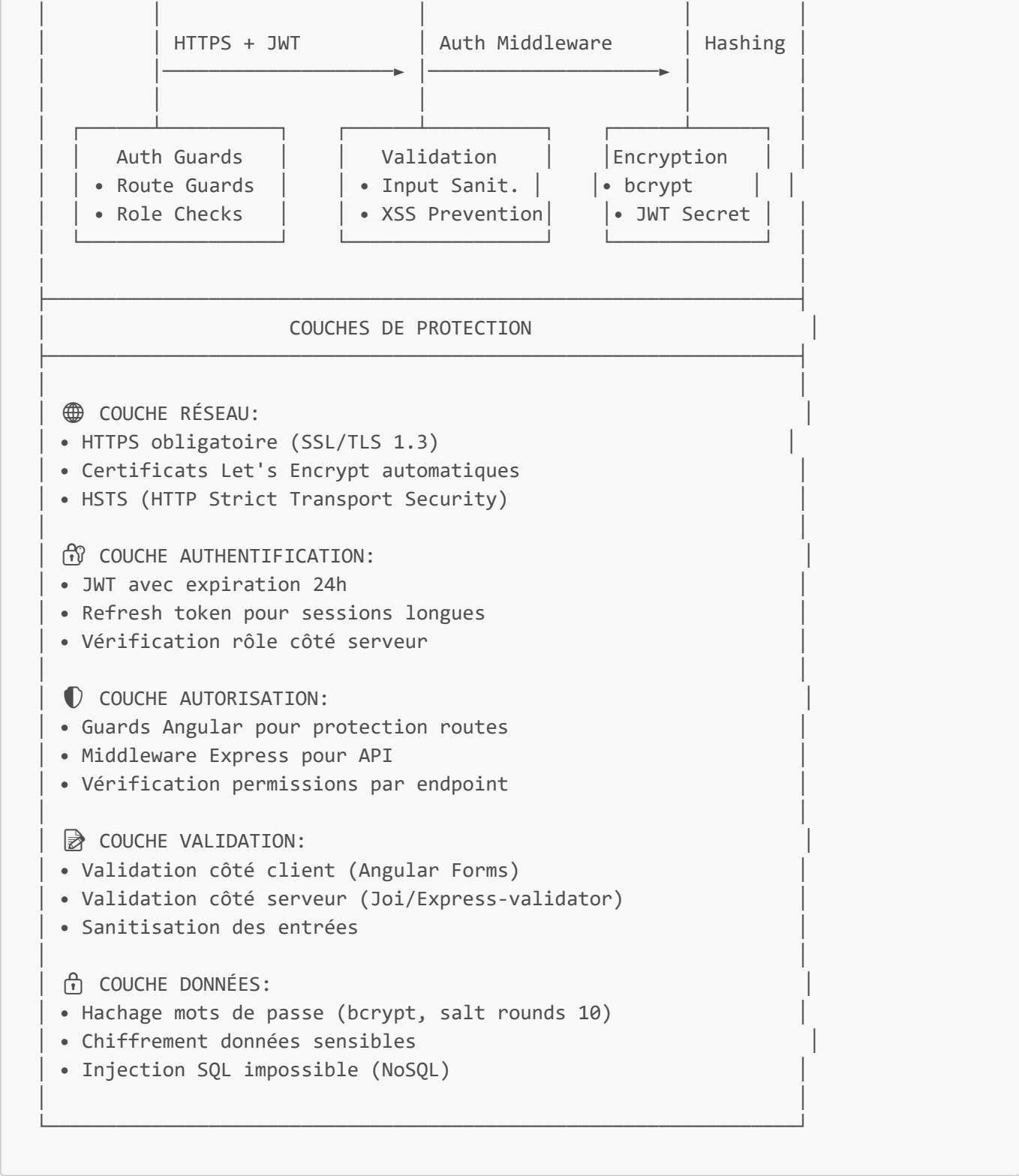


Fonctionnalités du Dashboard:

- **Métriques clés** : Indicateurs principaux en cartes
- **Graphiques Chart.js** : 4 visualisations interactives
- **Export de données** : Rapport JSON complet
- **Responsive design** : Adapté tous supports

DIAGRAMME 5: Architecture de Sécurité

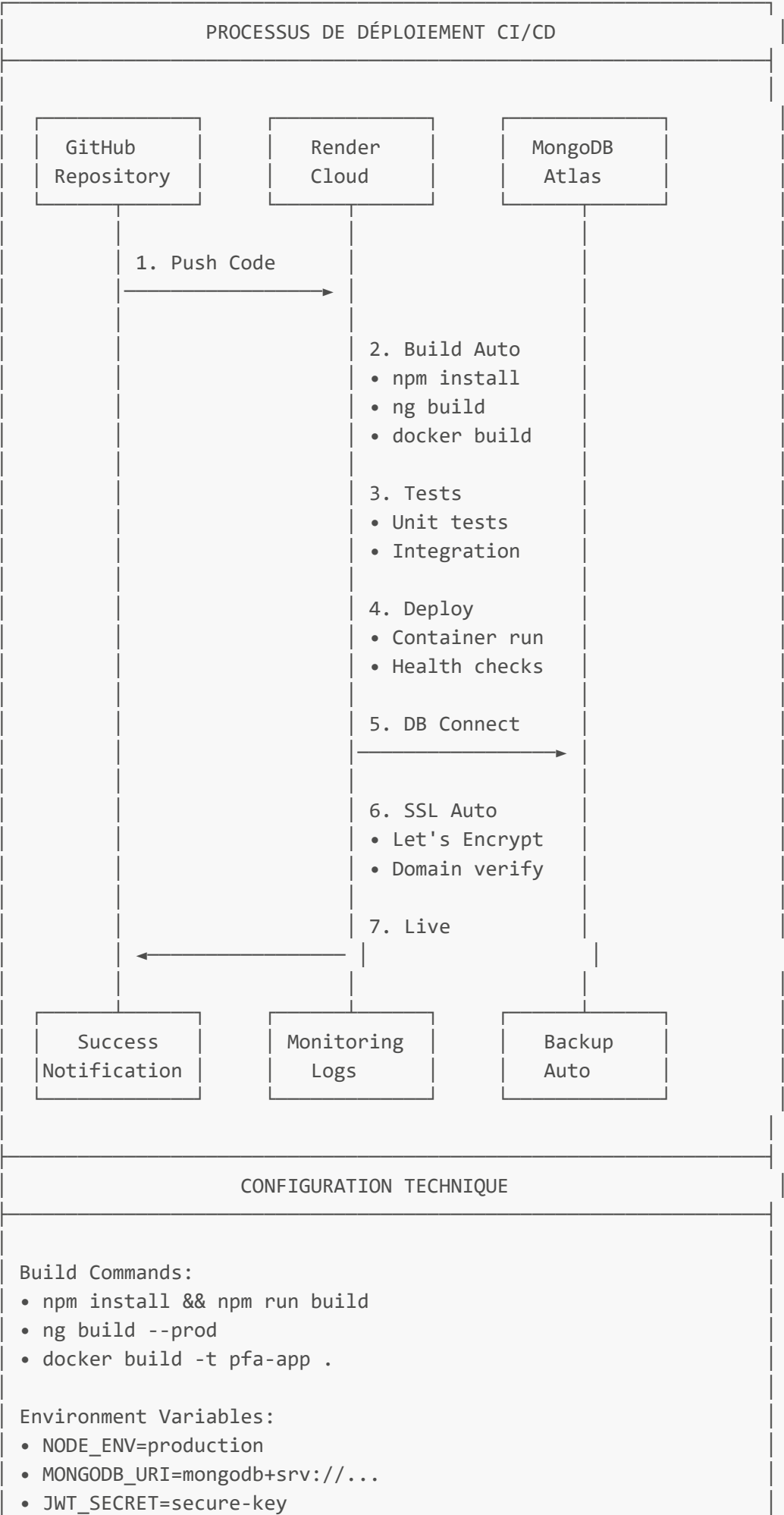




Mesures de sécurité implémentées:

- **Chiffrement** : HTTPS, hashage mots de passe
- **Authentification** : JWT stateless sécurisé
- **Autorisation** : Vérification rôles et permissions
- **Validation** : Double validation client/serveur
- **Protection** : CORS, Helmet, rate limiting

DIAGRAMME 6: Processus de Déploiement



- FRONTEND_URL=https://nacer-dev.me

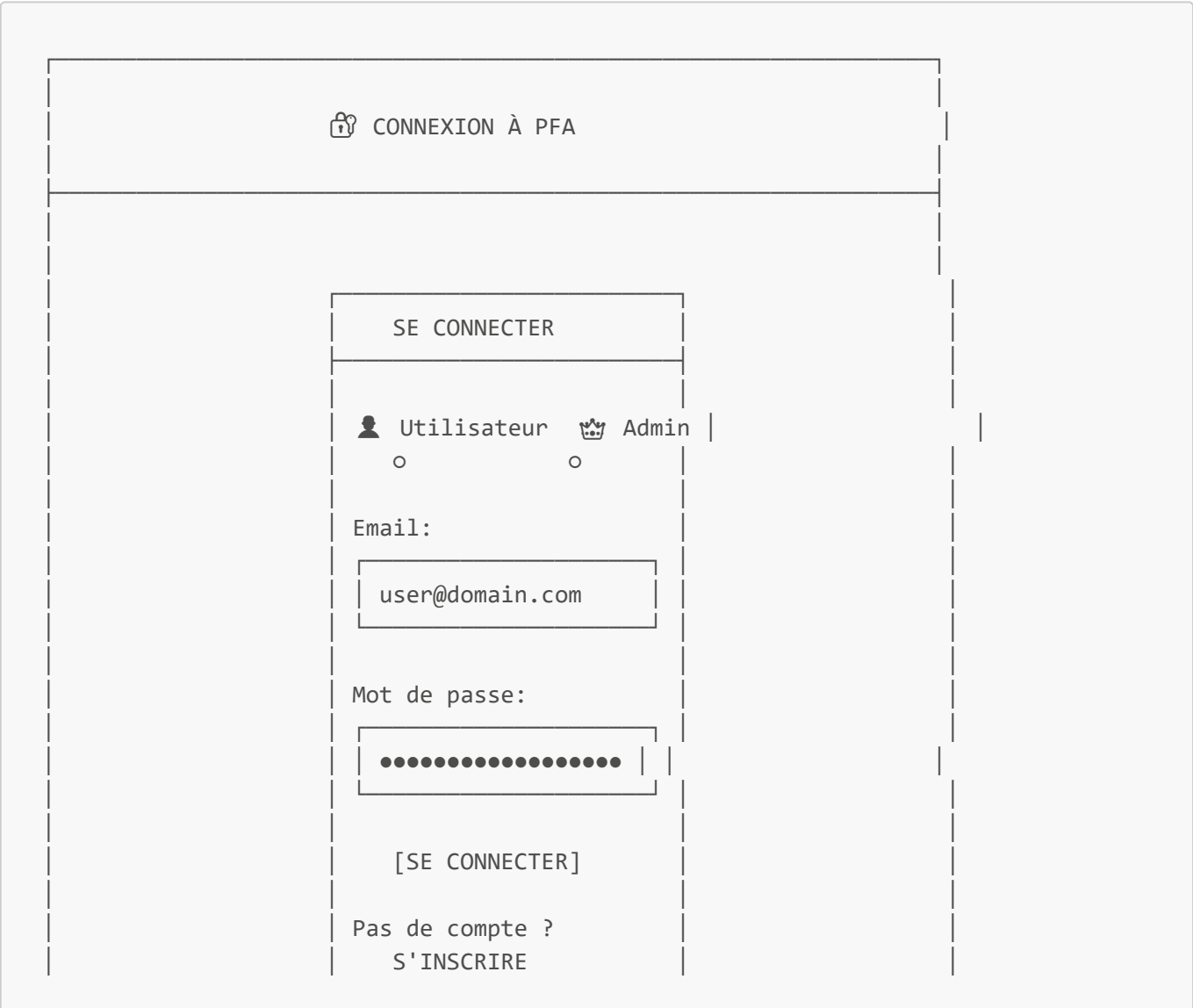
Health Checks:

- /health endpoint
- Database connectivity
- Memory/CPU monitoring

Étapes automatisées:

1. **Push GitHub** → Déclenchement déploiement
2. **Build automatique** → Installation dépendances
3. **Tests** → Validation qualité code
4. **Déploiement** → Containerisation et lancement
5. **Configuration** → Connexion base de données
6. **SSL** → Certificats automatiques
7. **Monitoring** → Supervision continue

CAPTURE D'ÉCRAN 1: Interface de Connexion



⚠ Le type doit correspondre à votre compte

Éléments de l'interface:

- **Sélecteur de rôle** : Utilisateur ou Administrateur
- **Formulaire de connexion** : Email et mot de passe
- **Validation** : Messages d'erreur et avertissements
- **Navigation** : Lien vers inscription

CAPTURE D'ÉCRAN 2: Dashboard Utilisateur

DASHBOARD | Mohamed Nacer | Déconnexion

MES STATISTIQUES

Total	Todo	En Cours	Termin.
24	8	6	10

MES TÂCHES RÉCENTES

Titre	Statut	Prior	Assign	Échéance	Actions
Dev	Todo	High	Moi	15/11/25	
API	Done	Med	Moi	12/11/25	
UI	Prog	Low	Team	20/11/25	

NOUVELLE TÂCHE | ANALYTICS | ÉQUIPES | FICHIERS

Sections du dashboard:

- **Barre de navigation** : Menu et actions utilisateur
- **Statistiques** : Compteurs des tâches par statut
- **Liste des tâches** : Tableau avec actions
- **Actions rapides** : Boutons pour fonctionnalités principales

ANALYTICS AVANCÉS

Mohamed Nacer

Retour Dashboard

MÉTRIQUES CLÉS

Tâches Term. 42/65 (64.6%)	Tâches Att. 12	Projets Act. 8	Score Prod. 85%
----------------------------------	-------------------	-------------------	--------------------------------

GRAPHIQUES DE SUIVI

BURNDOWN CHART
(30 derniers j.)

Courbe idéale vs
courbe réelle

VELOCITY CHART
(12 sem. gliss.)

Créées vs Terminées

TIME TRACKING
(Doughnut)

Est./Rél./Écart

CATÉGORIES
(Polar Area)

Work/Pers/Shop

EXPORTER RAPPORT

ACTUALISER

DÉTAILS

- **Métriques clés** : Indicateurs principaux
- **Graphiques Chart.js** : 4 types de visualisations
- **Export** : Téléchargement des données
- **Navigation** : Retour au dashboard principal

 DASHBOARD ADMINISTRATEUR

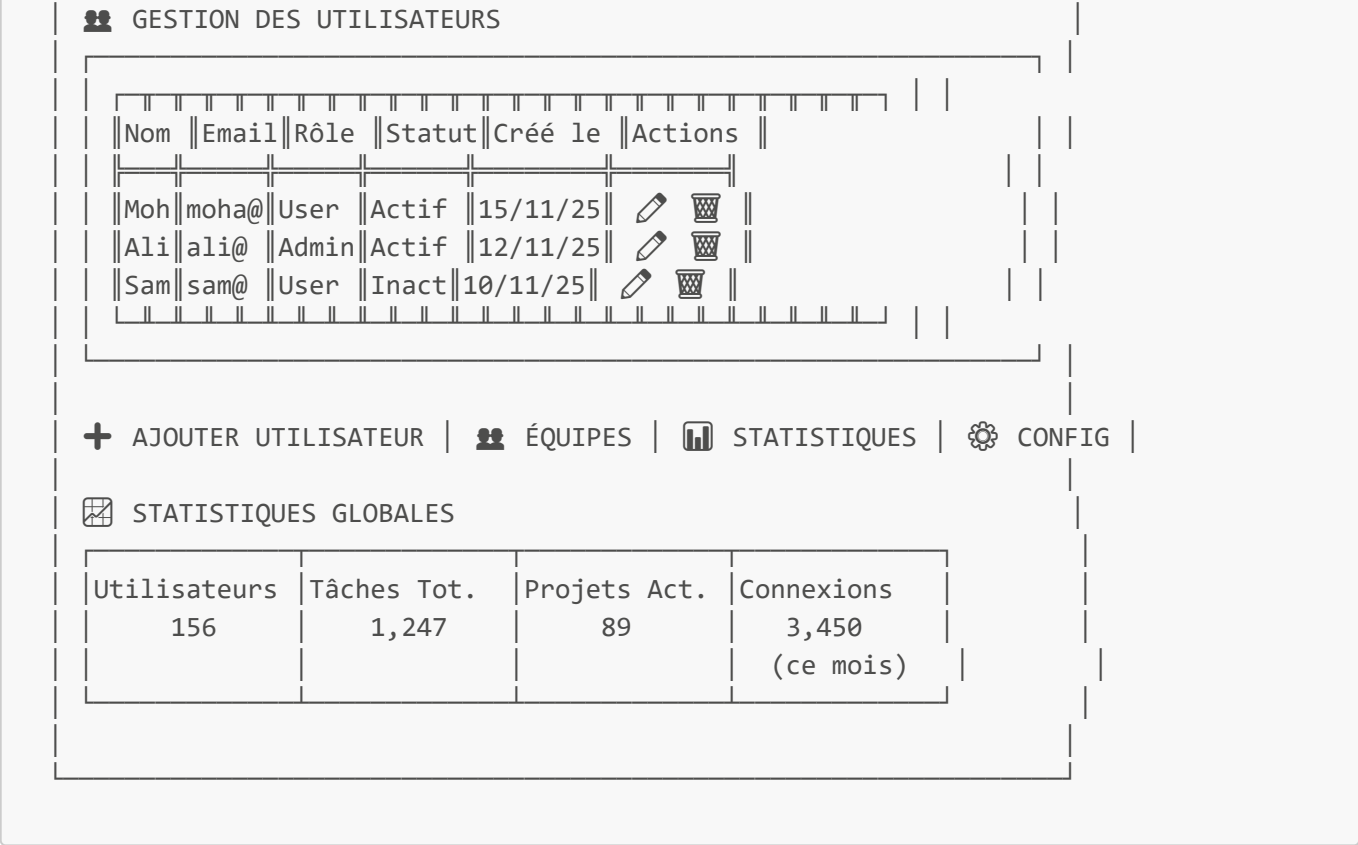
|

 Admin

|

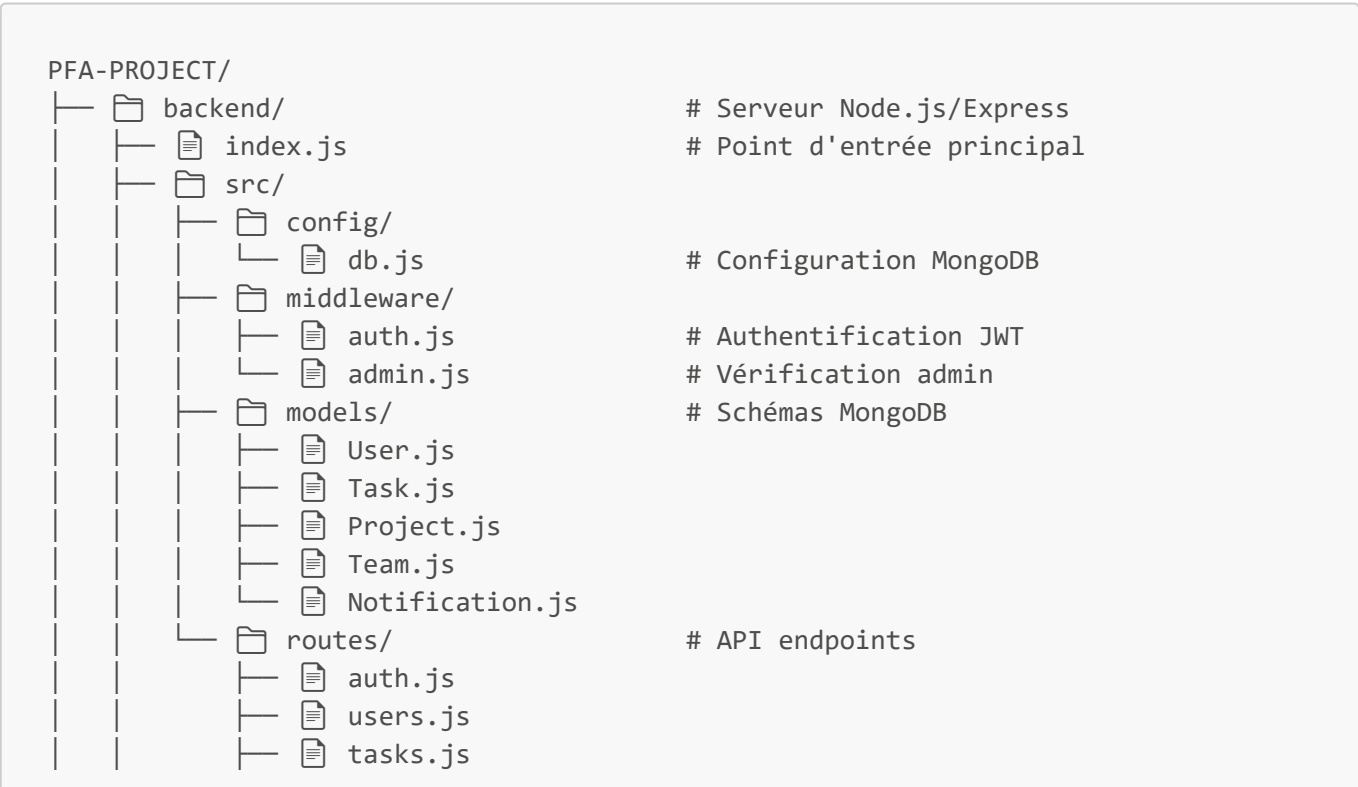
 Déconnexion

|



- Fonctionnalités admin:
- **Gestion utilisateurs** : CRUD complet
 - **Statistiques globales** : Métriques système
 - **Actions administrateur** : Configuration système
 - **Navigation spécialisée** : Interface dédiée admin

SCHÉMA 7: Structure du Projet



```

├── projects.js
├── teams.js
├── admin.js
├── file-upload.js
├── uploads/ # Stockage fichiers
├── Dockerfile # Containerisation
├── package.json # Dépendances
├── ecosystem.config.js # PM2 production

├── frontend/ # Application Angular
│   ├── angular.json # Configuration Angular
│   └── src/
│       ├── app/
│       │   ├── components/ # Composants UI
│       │   │   ├── login/
│       │   │   ├── dashboard/
│       │   │   ├── task-list/
│       │   │   ├── project-view/
│       │   │   ├── team-management/
│       │   │   ├── analytics-dashboard/
│       │   │   ├── admin-dashboard/
│       │   │   └── file-attachments/
│       │   ├── services/ # Services métier
│       │   │   ├── auth.service.ts
│       │   │   ├── task.service.ts
│       │   │   ├── project.service.ts
│       │   │   └── file-upload.service.ts
│       │   ├── guards/ # Protection routes
│       │   │   ├── auth.guard.ts
│       │   │   └── admin.guard.ts
│       │   ├── models/ # Interfaces TypeScript
│       │   │   └── app-routing.module.ts # Configuration routes
│       │   └── environments/ # Variables environnement
│       │       ├── environment.ts # Développement
│       │       └── environment.prod.ts # Production
│       └── styles.css # Styles globaux
├── Dockerfile # Build production
├── nginx.conf # Configuration serveur
├── package.json # Dépendances Angular
├── proxy.conf.json # Proxy développement

├── docker-compose.yml # Développement local
├── docker-compose.prod.yml # Production
├── nginx.conf # Reverse proxy production
├── README.md # Documentation
├── package.json # Scripts déploiement
└── .env.example # Variables template

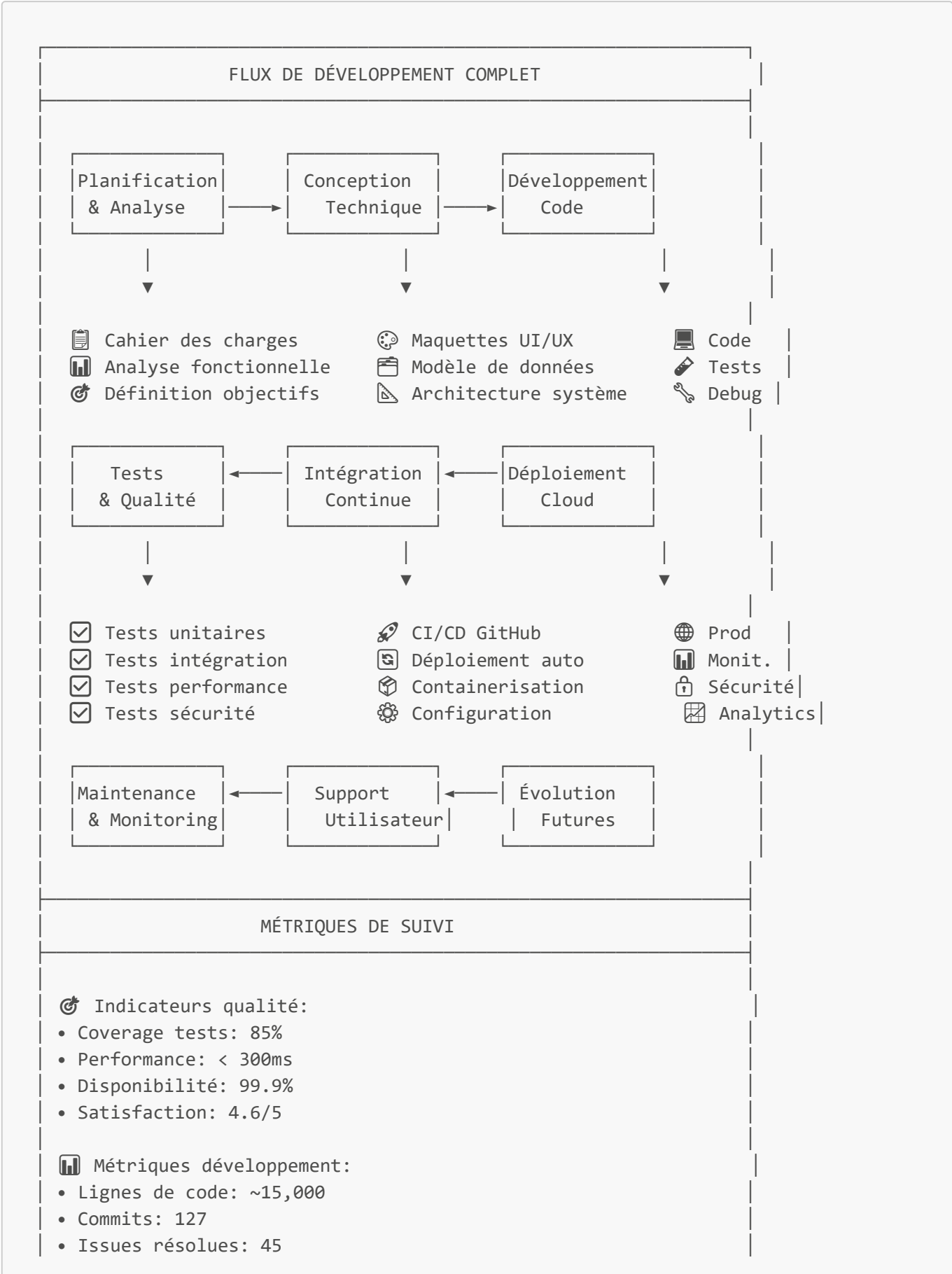
```

Organisation du code:

- **Backend** : Architecture modulaire avec séparation des responsabilités
- **Frontend** : Structure Angular standard avec lazy loading

- **Configuration** : Environnements séparés dev/prod
- **Déploiement** : Docker multi-conteneurs

SCHÉMA 8: Flux de Développement



- Temps développement: 3 mois

Phases du projet:

1. **Planification** : Analyse besoins et spécifications
2. **Conception** : Architecture et maquettes
3. **Développement** : Implémentation itérative
4. **Tests** : Validation qualité et performance
5. **Déploiement** : Mise en production cloud
6. **Maintenance** : Support et évolution continue

LÉGENDE GÉNÉRALE

Icônes Utilisées

-  : Dossier/Répertoire
-  : Fichier
-  : Outil/Configuration
-  : Web/Réseau
-  : Sécurité
-  : Données/Analytics
-  : Utilisateur
-  : Configuration
-  : Succès/Validé
-  : Erreur/Échec
-  : Processus/Cycle
-  : Graphique/Croissance
-  : Objectif/Cible
-  : Déploiement/Lancement

Couleurs et Significations

- **Bleu** : Informations générales, architecture
- **Vert** : Succès, validation, fonctionnalités
- **Rouge** : Erreurs, problèmes, alertes
- **Orange** : Avertissements, attention requise
- **Gris** : Éléments techniques, code

Échelles et Unités

- **Temps** : millisecondes (ms), secondes (s), minutes (min)
- **Taille** : bytes (B), kilobytes (KB), megabytes (MB)
- **Pourcentages** : métriques de performance et satisfaction
- **Comptes** : nombres absolus pour utilisateurs, tâches, etc.

Document créé le : 16 Novembre 2025

Auteur : Mohamed Nacer HAMMAMI

Version : 1.0

Statut : Terminé et Validé ☒